RequestsManagement – תיעוד מערכת

מבחן – Full Stack מערכת לניהול פניות

1. תיאור המערכת

פיתחתי מערכת לניהול פניות המתקבלות מארגונים ומעסיקים.

המערכת מאפשרת חיפוש וסינון פניות, מיון ודפדוף, עדכון סטטוס לפנייה בודדת או לקבוצת פניות, צפייה בהיסטוריית שינויים והצגת נתונים מסכמים.

המערכת פותחה לעבודה עם כמות גדולה של נתונים, ונעשה שימוש בכ־100,000 רשומות לצורך בדיקות.

2. טכנולוגיות

Backend: ASP.NET Core Web API, .NET 10 ,C# ו־Entity Framework Core 10.

Frontend: TypeScript, Angular, RxJS ו־Angular Material.

Database: SQL Server LocalDB.

3. מבנה הפרויקט

בצד השרת השתמשתי בחלוקה לשכבות:

- **Controllers** – קבלת בקשות מהלקוח והחזרת תשובות HTTP.
- **Services** – הלוגיקה העסקית ובדיקות תקינות.
- **Repositories** – ביצוע פעולות מול בסיס הנתונים.
- **Models ו־DTOs** – הגדרת הישויות והנתונים המועברים בין השרת ללקוח.
- **Data** – הגדרות בסיס הנתונים ו־Entity Framework Core.

החלוקה נועדה לשמור על קוד מסודר, הפרדת אחריות ותחזוקה נוחה.

4. שליפת נתונים

החיפוש, הסינון, המיון והדפדוף מתבצעים בצד השרת.

המערכת תומכת בסינון לפי סטטוס, עדיפות, שם ארגון, אחראי וטווח תאריכים, וכן בחיפוש לפי כותרת הפנייה ושם הארגון.

כדי למנוע טעינה מיותרת של נתונים, השרת מחזיר רק את הרשומות הנדרשות לעמוד הנוכחי.

קיימות גם בדיקות תקינות לפרמטרים והגבלה על מספר הרשומות בעמוד.

5. בסיס נתונים וביצועים

בחרתי ב־SQL Server וב־Entity Framework Core לצורך העבודה מול בסיס הנתונים.

הוספתי אינדקסים לשדות מרכזיים המשמשים בשליפה ובסינון, ובהם CreatedAt, Status, Priority, OrganizationName ו־AssignedTo.

ביצוע הסינון והדפדוף בבסיס הנתונים מאפשר לעבוד עם כמות גדולה של פניות בלי להעביר את כל הנתונים ל־Angular.

6. עדכון סטטוס ו־Concurrency

המערכת מאפשרת עדכון סטטוס לפנייה בודדת.

לצורך טיפול בעדכונים מתחרים השתמשתי ב־RowVersion. כאשר מתבצע ניסיון לעדכן רשומה שהשתנתה בינתיים, השרת מחזיר שגיאת Conflict מסוג HTTP 409.

כך ניתן למנוע מצב שבו עדכון של משתמש אחד דורס שינוי שביצע משתמש אחר.

7. היסטוריית שינויים

כל שינוי סטטוס נשמר בטבלת היסטוריה הכוללת:

- מזהה הפנייה.
- הסטטוס הקודם והסטטוס החדש.
- תאריך השינוי.
- הגורם שביצע את השינוי.

בנוסף, קיים API לשליפת היסטוריית השינויים של פנייה.

8. עדכון קבוצתי

מימשתי אפשרות לעדכון מספר פניות בבקשה אחת, בהתאם לדרישות המבחן.

בנוסף, הוספתי אפשרות לעדכון כל הפניות שעונות לתנאי סינון מסוימים.

בתחילה השתמשתי בעדכון בקבוצות באמצעות Entity Framework Core. לאחר שנתקלתי בבעיות ביצועים בכמות גדולה של נתונים, שיניתי את המימוש לפעולת SQL ייעודית.

העדכון ההמוני ושמירת ההיסטוריה מתבצעים באותה טרנזקציה, כדי לשמור על עקביות הנתונים.

9. נתונים מסכמים ו־Cache

המערכת מציגה נתונים מסכמים לפי סטטוס ועדיפות.

השתמשתי ב־MemoryCache עבור נתוני הסטטיסטיקה, עם זמן תפוגה של 60 שניות.

לאחר עדכון נתונים מתבצע ניקוי של ה־Cache, כדי שהנתונים המסכמים יחושבו מחדש.

אם המערכת תעבוד בעתיד עם מספר מופעי שרת, אפשר יהיה להשתמש ב־Distributed Cache כדי לשמור על עקביות בין השרתים.

10. ממשק Angular

הממשק כולל טבלת פניות, חיפוש, סינון, מיון, דפדוף, עדכון סטטוסים והצגת היסטוריית שינויים.

התקשורת עם השרת מתבצעת באמצעות Services ו־RxJS.

הממשק כולל טיפול במצבי טעינה, הודעות שגיאה ורענון הנתונים לאחר עדכון.

11.בדיקות

לפרויקט נפתח פרויקט בדיקות אוטומטיות נפרד.

הבדיקות מיועדות לכסות תרחישים של פעולה תקינה ,Validation ,עדכונים מתחרים ובדיקת.Integration

בנוסף, ביצעתי בדיקות ידניות של החיפוש, הסינון, הדפדוף, עדכון הסטטוסים, היסטוריית השינויים ורענון הנתונים.

יש לוודא שכל הבדיקות האוטומטיות עוברות לפני ההגשה.

12.מדידת ביצועים

במסגרת בדיקת הביצועים יש לתעד לפחות שתי פעולות API מול בסיס נתונים הכולל 100,000 פניות.

פעולה	**זמן תגובה**
שליפת פניות עם סינון ודפדוף	להשלמה לאחר מדידה
שליפת נתונים מסכמים	להשלמה לאחר מדידה

יש לצרף גם תוכנית ביצוע של שאילתה או מדד ביצועים מקביל.

צוואר בקבוק אפשרי הוא עדכון קבוצתי גדול, שעלול לגרום לנעילות במסד הנתונים. שיפור אפשרי הוא ביצוע עדכונים גדולים באמצעות תהליך רקע.

13.החלטות טכנולוגיות

ביצוע סינון ודפדוף בצד השרת

בחרתי לבצע את הפעולות בבסיס הנתונים במקום לטעון את כל הרשומות לדפדפן. כך ניתן לעבוד בצורה יעילה יותר עם כמות גדולה של נתונים.

שימוש ב־ SQLלעדכון המוני

בחרתי להשתמש בפקודת SQL ייעודית במקום לעדכן רשומות אחת־אחת באמצעות Entity Framework Core, בעקבות בעיות ביצועים שהתגלו במהלך העבודה.

14.הוראות הרצה

דרישות מקדימות

- .NET 10 SDK
- SQL Server LocalDBאוSQL Server
- Node.jsוnpm־
- Visual Studioאו סביבת פיתוח מתאימה

Backend

1. לפתוח את פתרון הפרויקט ב־Visual Studio.
2. לוודא שמחרוזת החיבור ב־ appsettings.json מתאימה לבסיס הנתונים המקומי.
3. להחיל את ה־ Migrations וליצור את נתוני הבדיקה באמצעות מנגנון ה־ Seed בפרויקט.
4. להריץ את השרת.

במהלך הפיתוח השרת פעל בכתובת:

https://localhost:7094בסביבת Development ניתן לבדוק את ה־ APIבאמצעות Swagger, אם הוא מופעל.

Frontend

לפתוח טרמינל בתיקיית Angular ולהריץ:

npm install

npm start

יש לוודא שכתובת ה־ APIבממשק תואמת לכתובת השרת.

בדיקות אוטומטיות

מתוך תיקיית הפתרון להריץ:

dotnet test

ולוודא שכל הבדיקות עוברות.

15. שימוש בכלי AI

במהלך הפיתוח נעזרתי בכלי AI לצורך חשיבה על פתרונות, כתיבת חלקים מהקוד, איתור שגיאות ושיפור ביצועים.

ביצעתי התאמות ידניות בהתאם למבנה הפרויקט ובדקתי את התוצאות באמצעות הרצה ובדיקות.

בין היתר, שיניתי ושיפרתי את מנגנון העדכון ההמוני לאחר שהתגלו בעיות בביצועים ובשמירת היסטוריית השינויים.